

Implementación de Estructuras de Datos Básicas (Pila, Cola y Lista Enlazada)

Alessandro Ivan Pachari Cahui
Nestor Fernando Huaylla Benito
Rodrigo Josseph Apaza Calizaya

8 de mayo de 2026

Introducción

LOS PROGRAMAS INFORMÁTICOS suelen operar en tablas de información. En la mayoría de los casos estas tablas no son simplemente masas amorfas de valores numéricos; implican importantes relaciones estructurales entre los elementos de datos. En su forma más simple, una tabla podría ser una lista lineal de elementos, cuando es Las propiedades estructurales relevantes podrían incluir las respuestas a preguntas como: ¿Qué elemento está primero en la lista? ¿Cuál es el último? ¿Qué elementos preceden y seguir una determinada? ¿Cuántos elementos hay en la lista? Se puede decir mucho sobre estructura incluso en este caso aparentemente simple:

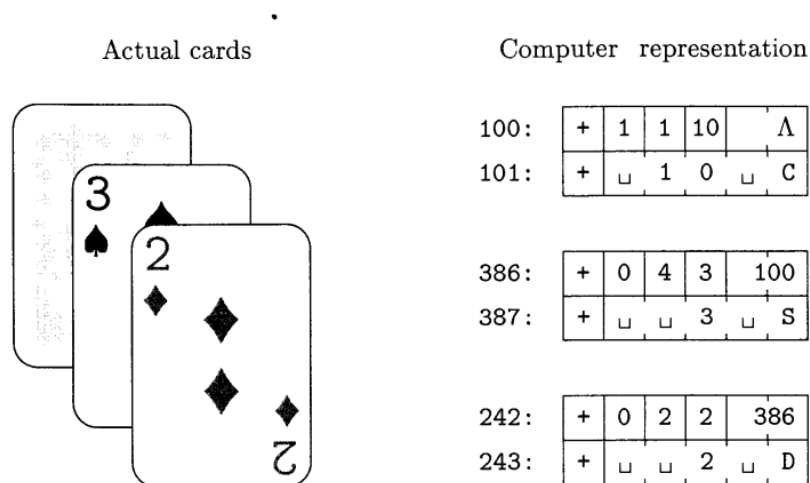


Figura 1

LOS datos suelen tener mucha más información estructural de la que realmente queremos representar directamente en una computadora. Por ejemplo, cada nodo de "tarjeta de juego.^{en} el la sección anterior tenía un campo SIGUIENTE para especificar qué tarjeta estaba debajo de ella en la pila, pero no proporcionamos una forma directa de averiguar qué tarjeta, si la hubiera, estaba encima de una determinada tarjeta, o para averiguar en qué pila estaba una tarjeta dada. Y, por supuesto, la suprimimos totalmente. la mayoría

de las características de las cartas reales: los detalles del diseño en la parte posterior, la relación con otros objetos de la sala donde se estaba jugando jugado, las moléculas individuales dentro de las cartas, etc

Parte A: Implementación de una Pila

Descripción

La pila es una estructura de datos que sigue el principio **LIFO** (**Last In, First Out**), donde el último elemento en ingresar es el primero en salir.

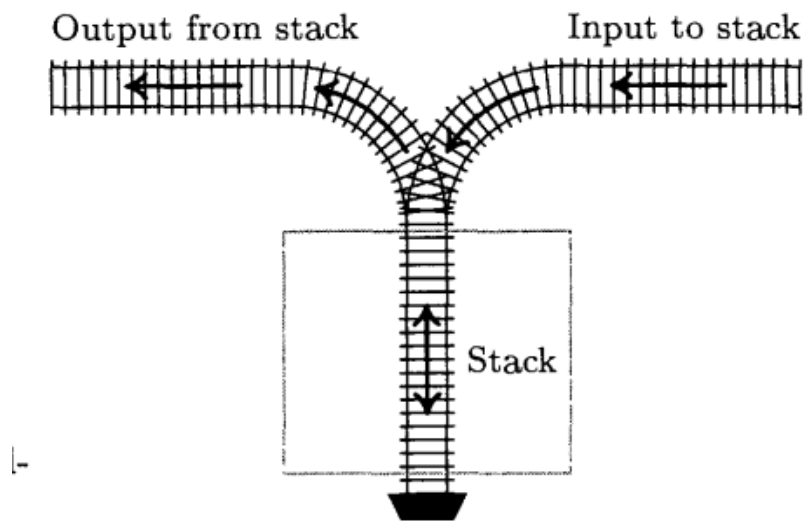


Figura 2: Una pila representada como una red de conmutación ferroviaria.

Muchas personas que se han dado cuenta independientemente de la importancia de las pilas y las colas les han dado nombres o por: Stacks se han llamado listas push-down, almacenes de reversión, sótanos, almacenes de anidación, pilas, listas de último ingreso por orden de salida ("LIFO"), e incluso listas de yo-yo. Las colas a veces se denominan tiendas circulares o listas de primera entrada ("FIFO").

Operaciones realizadas

- Insertar: 5, 10, 15, 20, 25
- Eliminar: 2 elementos
- Mostrar el tope
- Recorrer la pila

Código en C++

```

1
2 #define MAX 5
3
4 class Pila {
5 private:
6     int datos[MAX];
7     int tope;
8 public:
9     Pila() { tope = -1; }
10
11     void push(int valor) {
12         if (tope == MAX - 1) {
13             cout << "Error: Pila llena (overflow)" << endl;
14             return;
15         }
16         tope++;
17         datos[tope] = valor;
18     }
19
20     void pop() {
21         if (tope == -1) {
22             cout << "Error: Pila vacia (underflow)" << endl;
23             return;
24         }
25         tope--;
26     }
27
28     int top() {
29         if (tope == -1) {
30             cout << "Error: Pila vacia" << endl;
31             return -1;
32         }
33         return datos[tope];
34     }
35 };

```

Código en Python

```

1 class Pila:
2
3     def __init__(self, maximo):
4         self.datos = [0]*maximo
5         self.tope = -1
6
7     def push(self, valor):
8         if self.tope == len(self.datos) - 1:
9             print("Error: Pila llena (overflow)")
10            return
11            self.tope += 1
12            self.datos[self.tope] = valor
13
14    def pop(self):
15        if self.tope == -1:
16            print("Error: Pila vacia (underflow)")
17            return
18            self.tope -= 1
19

```

```

20     def top(self):
21         if self.tope == -1:
22             print("Error: Pila vacía")
23             return None
24         return self.datos[self.tope]

```

Complejidad

- push $\rightarrow O(1)$
- pop $\rightarrow O(1)$
- top $\rightarrow O(1)$
- recorrido $\rightarrow O(n)$

Ejecución paso a paso

Estado 1: Pila vacía



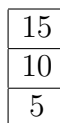
Estado 2: Insertar 5



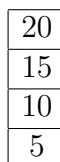
Estado 3: Insertar 10



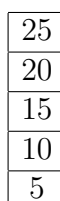
Estado 4: Insertar 15



Estado 5: Insertar 20



Estado 6: Insertar 25



Estado 7: Eliminar (pop)

20
15
10
5

Estado 8: Eliminar (pop)

15
10
5

Diagrama

ANTES (pila llena):

```
| 25 |  
| 20 |  
| 15 |  
| 10 |  
| 5  |
```

DESPUÉS:

```
| 15 |  
| 10 |  
| 5  |
```

Evidencia de ejecución en C++

```
Insertado: 5
Insertado: 10
Insertado: 15
Insertado: 20
Insertado: 25
Eliminado: 25
Eliminado: 20

Contenido de la pila:
| 15 |
| 10 |
| 5  |
-----

Presione una tecla para continuar . . .
```

Figura 3: Ejecución del programa de pila en C++.

Nota APA: Elaboración propia.

Resultado Final

Después de insertar y eliminar:

- Elementos eliminados: 25 y 20
- Tope final: 15
- Pila final: 15, 10, 5

Análisis

La pila permite un acceso rápido al último elemento insertado. Es útil en aplicaciones como manejo de llamadas recursivas y evaluación de expresiones.

Parte B: Implementación de una Cola

Descripción

La cola sigue el principio **FIFO (First In, First Out)**, donde el primer elemento en entrar es el primero en salir.

Operaciones realizadas

- Insertar: 3, 6, 9, 12
- Eliminar: 1 elemento
- Mostrar el frente
- Recorrer la cola

Código en C++

```
1 #define MAX 5
2
3 class Cola {
4 private:
5     int datos[MAX];
6     int frente, final;
7 public:
8     Cola() {
9         frente = 0;
10        final = -1;
11    }
12
13    void enqueue(int valor) {
14        if (final == MAX - 1) {
15            cout << "Error: Cola llena (overflow)" << endl;
16            return;
17        }
18        final++;
19        datos[final] = valor;
20    }
21
22    void dequeue() {
23        if (frente > final) {
24            cout << "Error: Cola vacia (underflow)" << endl;
25            return;
26        }
27        frente++;
28    }
29
30    int front() {
31        if (frente > final) {
32            cout << "Error: Cola vacia" << endl;
33            return -1;
34        }
35        return datos[frente];
36    }
37 };
```

Código en Python

```
1 class Cola:
2
3     def __init__(self, maximo):
4         self.datos = [0]*maximo
5         self.frente = 0
6         self.final = -1
7
8     def enqueue(self, valor):
9         if self.final == len(self.datos) - 1:
10             print("Error: Cola llena (overflow)")
11             return
12         self.final += 1
13         self.datos[self.final] = valor
14
15     def dequeue(self):
16         if self.frente > self.final:
17             print("Error: Cola vacia (underflow)")
18             return
19         self.frente += 1
20
21     def front(self):
22         if self.frente > self.final:
23             print("Error: Cola vacia")
24             return None
25         return self.datos[self.frente]
```

Complejidad

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(1)$
- front $\rightarrow O(1)$
- recorrido $\rightarrow O(n)$

Ejecución paso a paso

Estado 1: Cola vacía

0	1	2	3

Estado 2: Insertar 3

0	1	2	3
3			

Estado 3: Insertar 6

0	1	2	3
3	6		

Estado 4: Insertar 9

0	1	2	3
3	6	9	

Estado 5: Insertar 12

0	1	2	3
3	6	9	12

Estado 6: Eliminar (dequeue)

0	1	2	3
	6	9	12

Diagrama

ANTES:

[3] -> [6] -> [9] -> [12]

DESPUÉS:

[6] -> [9] -> [12]

Evidencia de ejecución en C++

```
Insertado: 3
Insertado: 6
Insertado: 9
Insertado: 12
Eliminado: 3

Contenido de la cola:
6 9 12

Presione una tecla para continuar . . .
```

Figura 4: Ejecución del programa de cola en C++.

Nota APA: Elaboración propia.

Resultado Final

- Elemento eliminado: 3
- Frente final: 6
- Cola final: 6, 9, 12

Análisis

Las colas son utilizadas en sistemas donde el orden de atención es importante, como colas de impresión o sistemas de atención al cliente.

Parte C: Lista Enlazada Simple

Descripción

Una lista enlazada está formada por nodos, donde cada nodo contiene un dato y una referencia al siguiente nodo.

Operaciones realizadas

- Insertar al inicio: 8, 4
- Insertar al final: 11
- Eliminar el primer nodo
- Mostrar la lista

Código en C++ (Inserción y Liberación)

```
1
2 struct Nodo {
3     int dato;
4     Nodo* siguiente;
5 };
6
7 // Insertar al inicio: O(1)
8
9 void insertarInicio(Nodo*& cabeza, int valor) {
10     Nodo* nuevo = new Nodo();
11     nuevo->dato = valor;
12     nuevo->siguiente = cabeza;
13     cabeza = nuevo;
14 }
15
16 // Eliminar al inicio: O(1) y liberacion de memoria
17 void eliminarInicio(Nodo*& cabeza) {
18     if (cabeza != nullptr) {
19         Nodo* temporal = cabeza;
20         cabeza = cabeza->siguiente;
21         delete temporal; // Control explicito de memoria
```

```

22     }
23 }

```

Análisis de Memoria

A diferencia de Python, donde la recolección de basura es automática[cite: 191], en C++ se utiliza el operador `delete` para liberar el espacio ocupado por el nodo eliminado. Esto garantiza un control detallado de los recursos, un aspecto evaluado fundamental en esta práctica[cite: 162].

Código en Python

```

1
2 class Nodo:
3     def __init__(self, dato):
4         self.dato = dato
5         self.siguiente = None

```

Análisis de Complejidad Temporal

- **Insertar al inicio:** $O(1)$ [cite: 185].
- **Insertar al final:** $O(n)$, ya que requiere recorrer la lista hasta el último nodo[cite: 187]. *Nota: Podría optimizarse a $O(1)$ manteniendo un puntero al final.*
- **Eliminar inicio:** $O(1)$ [cite: 188].
- **Recorrido:** $O(n)$ [cite: 189].

Ejecución paso a paso

Estado 1: Lista vacía

NULL

Estado 2: Insertar al inicio (8)

8 → NULL

Estado 3: Insertar al inicio (4)

4 → 8 → NULL

Estado 4: Insertar al final (11)

4 → 8 → 11 → NULL

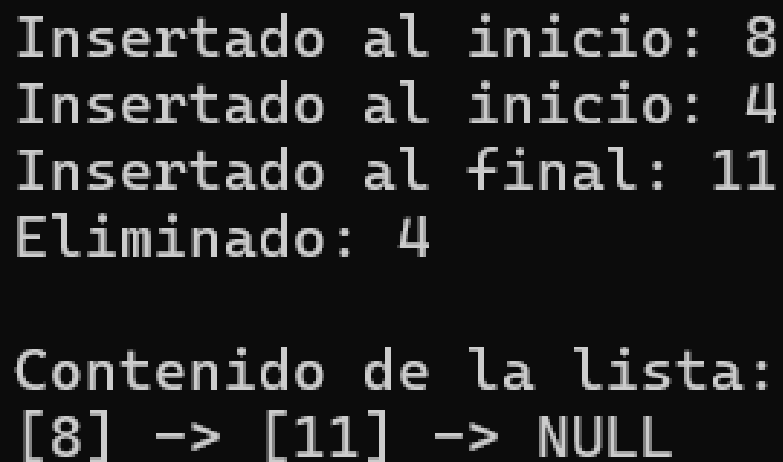
Estado 5: Eliminar inicio

$\boxed{8} \rightarrow \boxed{11} \rightarrow \text{NULL}$

Estado Final

- Nodo eliminado: 4
- Lista final: 8, 11

Evidencia de ejecución en C++



```
Insertado al inicio: 8
Insertado al inicio: 4
Insertado al final: 11
Eliminado: 4

Contenido de la lista:
[8] -> [11] -> NULL
```

Figura 5: Resultado de las operaciones realizadas sobre la lista enlazada simple.

Nota APA: Elaboración propia.

Diagrama

ANTES:

[4] -> [8] -> [11] -> NULL

DESPUÉS:

[8] -> [11] -> NULL

Resultado Final

- Nodo eliminado: 4
- Lista final: 8, 11

Análisis

Las listas enlazadas permiten inserciones y eliminaciones dinámicas sin necesidad de reorganizar la memoria, a diferencia de los arreglos.

Comparación entre C++ y Python

- C++ requiere manejo manual de memoria, mientras que Python lo hace automáticamente.
- Python tiene una sintaxis más simple y legible.
- C++ ofrece mayor control y eficiencia.
- Python permite desarrollar más rápido pero con menor control del hardware.

V. Comparación técnica de implementaciones

Basado en el desarrollo de las actividades, se presenta la siguiente comparativa técnica entre ambos entornos[cite: 191]:

Cuadro 1: *Diferencias estructurales entre entornos compilados y de scripting*

Aspecto evaluado	C++	Python
Control de memoria	Explícito (punteros, <code>new</code> , <code>delete</code>)	Implícito (recolección automática de basura)
Estructura de código	Más detallada y extensa	Más concisa y abstracta
Flexibilidad de listas	Requiere implementación manual	Uso directo de listas dinámicas (<code>list</code>)
Complejidad de implementación	Alta para estructuras dinámicas	Baja, pero menos transparente internamente
Tipado de variables	Estático	Dinámico

Conclusiones

- Se logró implementar manualmente las estructuras de datos básicas.
- Se comprendió el funcionamiento interno de pilas, colas y listas enlazadas.
- Se identificaron diferencias clave entre C++ y Python.
- Estas estructuras son fundamentales para el desarrollo de algoritmos eficientes.

Simulador utilizado

Para verificar visualmente el comportamiento de las estructuras de datos (pila, cola y lista enlazada), se utilizó un simulador interactivo disponible en línea.

- Simulador de estructuras de datos: https://nestorhuaylla.github.io/Simulator_of_linear_structures/

Referencias

- [1] Knuth, D. E. (1997). *The Art of Computer Programming: Vol. 1. Fundamental Algorithms* (3.^a ed.). Addison-Wesley.
- [2] Joyanes Aguilar, L., & Zahonero Martínez, I. (2007). *Estructuras de datos en C++* (2.^a ed.). McGraw-Hill.